



IUT SAINT-DIÉ-DES-VOSGES
INFORMATIQUE
SAE 1.01

Implémentation d'un besoin client

(Rapport)

Enseignants : BOURJIJ Abdellatif
KERRIEN Erwan

Etudiants : LIGEIRO, Eunice Eugenio
MECHISSO, Wendy Patrícia De Oliveira

Saint-Dié-Des-Vosges, Janvier 2025

Indice

1. INTRODUCTION	2
2. STRUCTURES DE DONNEES ET FONCTIONS IMPLEMENTEES	2
2.1 Module Location	2
2.2 Module Stack.....	2
2.3 Intégration avec le Module Mobile	3
2.4 Impact sur les Fonctions de Jeu.....	3
3. FONCTIONNALITES ET LIMITATIONS	3
3.1 Fonctionnalités.....	3
3.2 Limitations.....	3
4. REPARTITION DES TACHES	4
5. CONCLUSION.....	4

1. INTRODUCTION

Ce rapport vise à décrire les choix de conception et les fonctionnalités du programme développé dans le cadre du projet, tout en identifiant ses limites et en précisant la répartition des tâches entre les membres de l'équipe.

2. STRUCTURES DE DONNEES ET FONCTIONS IMPLEMENTEES

2.1 Module Location

Le module **Location** est conçu pour représenter les différents lieux du jeu et leurs relations. Chaque localisation peut contenir des informations liées à son environnement et aux directions possibles pour passer à d'autres localisations.

➤ Fonctions clés :

- LocationNew : Crée une localisation en mémoire avec les attributs spécifiés.
- LocationDelete : Libère la mémoire allouée à une localisation.
- LocationPrint : Affiche les informations associées à une localisation.
- LocationInit : Initialise toutes les localisations et les relie selon la carte prévue pour le jeu.

Justification : Ce module permet une gestion centralisée et efficace des lieux, facilitant leur création, leur destruction, et leur manipulation par le joueur.

2.2 Module Stack

Le module *Stack* fournit une structure de pile utilisée pour gérer dynamiquement les ressources liées aux localisations, notamment lors de leur destruction.

➤ Fonctions clés :

- StackPush : Ajoute une localisation à la pile.
- StackPop : Retire la localisation au sommet de la pile.
- StackHead : Accède à l'élément en haut de la pile sans le retirer.

Justification : Une pile permet de gérer les localisations dans un ordre LIFO (*Last In, First Out*), idéal pour garantir une libération ordonnée de la mémoire.

2.3 Intégration avec le Module Mobile

Pour permettre au joueur de se déplacer entre les localisations, la structure *Mobile* a été modifiée pour inclure un pointeur vers une localisation. La commande *MobileMove* permet de changer dynamiquement la position du joueur.

2.4 Impact sur les Fonctions de Jeu

Les fonctions *GameInit* et *GameShutdown* ont été modifiées pour initialiser les localisations et garantir une libération correcte des ressources en utilisant la pile.

3. FONCTIONNALITES ET LIMITATIONS

3.1 Fonctionnalités

- **Déplacement du joueur :** Le joueur peut se déplacer entre les localisations grâce aux fonctions intégrées au module *Mobile*.
- **Affichage des informations :** Les localisations peuvent afficher leurs attributs via *LocationPrint*.

3.2 Limitations

- **Complexité croissante :** Avec l'augmentation du nombre de localisations ou de connexions, le programme pourrait devenir plus difficile à optimiser.
- **Tests non exhaustifs :** Certaines situations extrêmes ou cas limites pourraient ne pas être couvertes par les tests réalisés.

4. REPARTITION DES TACHES

La répartition des tâches était la suivante :

➤ **Wendy Mechisso :**

- Création des fonctions pour le module *Location* (LocationNew, LocationDelete, LocationPrint).
- Implémentation de LocationInit et des connexions entre les localisations.
- Implémentation de la structure *Stack* et de ses fonctions.

➤ **Eunice Ligeiro:**

- Modification des fonctions de commande (cmdLook, cmdGo) et gestion des mouvements du joueur.
- Gestion des aspects mémoire et des intégrations.
- Participation aux tests et à la validation.

5. CONCLUSION

Le projet a permis de concevoir un système solide pour gérer les localisations d'un jeu, en s'appuyant sur des structures de données adaptées comme les piles. Bien que les fonctionnalités principales soient réalisées avec succès, des optimisations pourraient encore être apportées pour améliorer les performances et la scalabilité. Les tests ont confirmé la fiabilité des fonctions principales, mais une couverture plus large serait souhaitable pour garantir une robustesse totale.